

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Software Architecture and Design Specification

Part: Architecture and Design (Sections 3 and 4)

Hostel Management System

Project: Hostel Management System

Version: 1.1

Group: 3

Date: 07-10-2026

Status: Draft for team review

Name	SRN
Raina Kareem	PES1UG24CS361
Prema P Kotur	PES1UG24CS343
Praakruthi PS	PES1UG24CS324
Ojas Taori	PES1UG24CS308

Revision History

Version	Date	Author	Change Summary
1.0	05-10-2026	Group 3 (design)	Design section of the SAD for Mini-Project Deliverables Part-1: design overview, UML sequence diagrams SD-1 to SD-4, API design, error handling, logging and monitoring, UX design, open issues. Written against SRS Version 1.0 dated 04-10-2026.
1.1	07-10-2026	Group 3 (architecture)	Added Section 3 Architecture (goals and constraints, stakeholders, component diagram, component descriptions, pattern and rationale, technology stack and data stores, risks, traceability, security architecture) in front of Section 4. Design figures renumbered from Figure 1 to 4 into Figure 4.1 to 4.4. Section 4 text is otherwise unchanged.

Approvals

Role	Name	Signature/Date
Team Lead	_____	_____
Faculty Evaluator	_____	_____

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Hostel Management System. It describes the major system components, their interactions, design decisions, data handling, security measures, and detailed workflows required to implement the system.

1.2 Scope

The Hostel Management System manages student registration and profiles, hostel and room management, room applications and allocation, room changes and check-out, complaints and maintenance, fee records and payments, reports, notifications, and announcements.

1.3 Audience

This document is intended for the development team, testers, project evaluators, hostel administrators and wardens, and other stakeholders involved in the development, operation, and review of the system.

1.4 Definitions

HMS, SRS, SAD, SRN, C1–C8, RBAC, ORM, SMTP, TLS, and DPDP.

2. Document Overview

2.1 How to use this document

This document describes the architecture and detailed design of the Hostel Management System. Section 3 presents the system architecture, including goals and constraints, stakeholders, UML component and deployment views, component descriptions, architecture pattern, technology stack, risks, requirement traceability, and security architecture. Section 4 presents the detailed design, including package structure, data design, state models, UML sequence diagrams, API design, error handling, logging and monitoring, UX design, and open issues.

2.2 Related Documents

Software Requirements Specification (SRS), Version 1.0

SAD Template

3. Architecture

3.1 Goals & Constraints

Goals

- Make double allocation of a bed impossible, using database constraints and not only application checks (FR-4.6, NFR-REL-1). Where goals conflict, allocation correctness is placed above speed (SRS Section 5.4).
- Bring the warden effort for one application down to under 10 minutes, with one searchable record per resident (SRS Section 1.3).
- Meet the performance targets: 95% of page requests within 2 s at 200 users, an allocation within 1 s, and Auto-Allocate of 500 applications within 60 s (NFR-PERF-1, 2 and 4).
- Grow to 5,000 students and 4,000 beds with no code change, only a bigger server (NFR-SCL-1).
- Be available 99% of the time between 07:00 and 23:00 IST each month, and 99.5% during the application window (NFR-AVL-1).
- Lose at most 24 hours of data and recover within 4 hours, using daily backups (NFR-SAFE-1).
- Protect the personal data of students (SEC-OBJ-1 to 5, DPDP Act 2023).

Constraints

- Python 3.12 with Django 5.x, server-rendered templates with Bootstrap 5 and no single-page-application framework (C-1, C-4).
- PostgreSQL 16, accessed only through the Django ORM with no hand-written SQL (C-2).
- Free and open-source libraries only (C-3).
- Deployable on one university virtual machine (C-6).
- Compliance with the DPDP Act 2023 (C-7).
- Built by four students in one semester (C-8).
- Assumptions that shape the design: the master list arrives as a CSV file (A-1), e-mail goes through the university SMTP relay (A-3), payments are only recorded and not collected (A-4), and each hostel is for one gender (A-5).

3.2 Stakeholders & Concerns

Stakeholder	Main concerns	Where the architecture answers them
Students (about 2,000)	Easy application, correct status, privacy of their data, use from a phone	Server-rendered responsive UI; scoped access (Section 3.9)
Wardens (about 10)	Live vacancies, no double booking, access only to their own hostels	C4 transaction and constraints; hostel scope checked in Core
Administrator (1 to 2)	Control of users, rooms, application windows and the audit log	C1, C3, Core audit
Hostel office	Accurate occupancy, dues and complaint reports	C7
University IT	One VM, TLS, SMTP relay, easy backup	Deployment view (Section 3.6)
Faculty and evaluators	Traceability from SRS to design	Section 3.8
Developers and testers	Clear modules and testable service interfaces	Sections 3.4 and 3.5

3.3 Component (UML) Diagram

Figure 3.1 shows the components of the HMS application and the main dependencies between them. The numbers on the arrows are explained in the table that follows the figure.

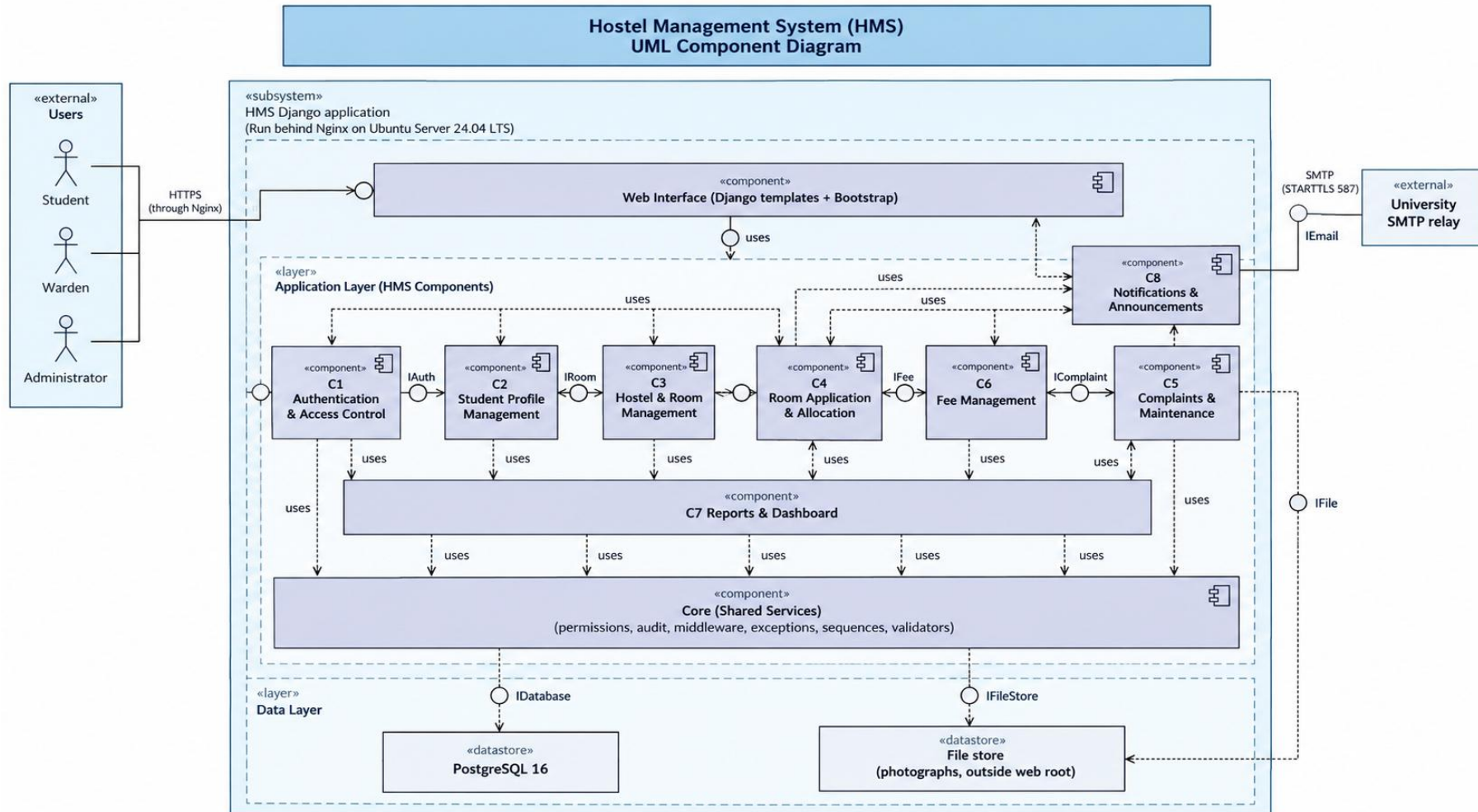


Figure 3.1: UML Component diagram of HMS

Dependencies shown in Figure 3.1

No.	Between	What is exchanged	Reference
1	C1 → C2	At registration C1 asks C2 to check the SRN against the master list.	FR-1.1; SD-3
2	C1 → C8	C1 queues the verification and password-reset e-mails.	FR-1.2, FR-1.6, FR-9.1
3	C4 ↔ C2	C4 reads gender, year, priority flag and profile completeness when applications are submitted and ordered. C2 reads the current hostel and room of a student for the search results.	FR-4.2, FR-4.4, FR-2.5, FR-2.6
4	C4 ↔ C3	C4 reads rooms and beds and has the room status recomputed through C3 after every allocation change. C3 checks for active allocations before a hostel or room is deactivated.	FR-3.5, Section 4.1.4
5	C4 ↔ C6	C4 raises the AllocationCreated signal, which makes C6 create the fee record. C6 calls AllocationService.confirm when the fee record becomes Paid.	FR-7.1, FR-7.3
6	C4 ↔ C8	C4 queues the application, allocation, rejection and room-change e-mails. C8 finds the residents of a hostel when an announcement is posted.	FR-4.10, FR-9.1, FR-9.3
7	C5 → C4	C5 reads the active allocation of the student to fill in hostel and room of a complaint.	FR-6.2
8	C5 → C8	C5 queues the e-mail for a complaint status change.	FR-9.1
9	C7 → C2, C3, C4, C5, C6	Read-only queries for the dashboards and the four reports.	FR-8.1 to FR-8.4
10	C1 to C8 → Core	Role and hostel checks, audit entries, error handling, document numbers and validators.	BR-1, SEC-REQ-8

The dashed arrows are UML dependencies: the component at the tail uses the component at the head. An arrow with a head at both ends means that each component uses the other. Core is drawn once, with one arrow, because all of C1 to C8 use it. Components call each other only through service interfaces (Section 4.1.1); the only exception is the AllocationCreated signal in arrow 5, which lets C6 react to a new allocation without C4 knowing anything about fees.

The browser, the SMTP relay, PostgreSQL and the file store are outside the application. The master-list and rooms CSV files reach the application as uploads by the Administrator through the browser (A-1), so no separate box is drawn for the university records system. Nginx and Unicorn are not components of the application and appear in the deployment view (Figure 3.2).

3.4 Component Descriptions

The SRS and Section 4 use the short names given in brackets. The Uses column agrees with the arrows in Figure 3.1.

Component	What it does	Owns	Uses
C1 Authentication (Auth), package accounts	Registration, e-mail verification, login with lock-out, password reset, 30-minute sessions, creation of Warden and Administrator accounts, role changes and deactivation.	User, AuthToken, AuthAttempt	C2, C8, Core

Component	What it does	Owns	Uses
C2 Student Profile (Profile), package students	Profile view and edit with validation, read-only fields taken from the master list, master-list CSV import, student search limited to the Warden's hostels, priority flag.	MasterListEntry, Student	C4 (read only), Core
C3 Hostel & Room Management (Rooms), package rooms	Hostels with gender designation and assigned Wardens, rooms with type, capacity and fee, availability grid, rooms CSV import, room status (Available, Full, Under Maintenance).	Hostel, HostelWarden, Room	C4 (read only), Core
C4 Room Application & Allocation (Allocation), package allocation	Application windows, applications, Warden queue, manual allocation, Auto-Allocate, rejection, room change, check-out, allocation history, cancellation of unpaid allocations.	ApplicationWindow, RoomApplication, Allocation, RoomChangeRequest, AutoAllocateRun	C2, C3, C6 (through the signal), C8, Core
C5 Complaints & Maintenance (Complaints), package complaints	Raising complaints with an optional photo, the Open, In Progress, Resolved or Rejected flow, reopening within 7 days, highlighting of complaints older than 72 hours.	Complaint	C4, C8, Core
C6 Fee Management (Fees), package fees	Fee record per allocation, payment recording, protection against duplicate payments, PDF receipts, list of Provisional allocations with no payment.	FeeRecord, FeePayment	C4, Core
C7 Reports & Dashboard (Reports), package reports	Three dashboards, four reports (Occupancy, Resident List, Fee Dues, Complaint Summary) and CSV and PDF export.	Nothing; it only reads	C2, C3, C4, C5, C6 (read only), Core
C8 Notifications & Announcements (Notifications), package notifications	Outbox queue and mail dispatcher with retries, hostel announcements with an expiry date.	NotificationOutbox, Announcement	C4 (read only), Core
Core shared services, package core	Not a feature. Role and hostel-scope checks (BR-1), audit writing, request middleware and error handler, exception hierarchy, document numbers (APP, CMP, RCP) and validators.	DocumentSequence, AuditLog	Nothing

Each component is a Django application with four layers of its own (models, services, forms, views), as described in Section 4.1.1. No component writes to the tables of another component.

3.5 Chosen Architecture Pattern and Rationale

HMS is a three-tier web application: the browser, the Django application tier and the PostgreSQL database, as stated in SRS Section 2.1. The application tier is a modular monolith. It is one Django project, deployed as one unit, and split into eight feature apps and a Core app, each with its own layers.

The reasons for this choice are:

- **One VM, one semester, four people (C-6, C-8).** A single deployable unit is what the team can build, test and run.
- **Several components change together.** One allocation changes the allocation, the room status, the fee record, the outbox and the audit log, and these must be committed together or not at all (FR-4.6, FR-5.2). Inside one process and one database this is an ordinary transaction.
- **The scale is modest.** About 2,000 students and 200 concurrent users fit the reference server (NFR-PERF-1), and NFR-SCL-1 asks only for a bigger server at 5,000 students.

- **The boundaries still exist.** The service-interface rule, the rule that no component writes to another component's tables, and the C1 to C8 split give separation of concerns and modules that can be tested one by one.

The alternatives that were rejected are:

- **Microservices.** Allocation and room change would need distributed transactions, and there would be several deployments to run, which is too much for this scale and team.
- **A single-page application.** Ruled out by constraint C-4.
- **A message broker for e-mail.** Not needed. The outbox table is the queue and systemd timers run the dispatcher (Section 4.1.7).

The price of this choice is that all components share one process and one database, so a bug in one of them can affect the others. The layering rules, code review and tests on the service interfaces are the control for this. There is also a two-way dependency between C4 and several other components (arrows 3 to 6 in Figure 3.1). It is kept manageable because the look-ups that go towards C4 are read-only and go through its service layer.

3.6 Technology Stack & Data Stores

Area	Choice	Source
Language and framework	Python 3.12, Django 5.x	C-1
Database	PostgreSQL 16 through psycopg 3, Django ORM only	C-2, SRS Section 3.2
Web server	Gunicorn 22.x behind Nginx 1.24+, TLS at Nginx	SRS Section 2.4
Operating system	Ubuntu Server 24.04 LTS	SRS Section 2.4
User interface	Django templates, Bootstrap 5.3 served locally	C-4, SRS Section 3.2
PDF	ReportLab 4.x for receipts and reports	D-1
E-mail	University SMTP relay, STARTTLS on port 587	A-3, COM-3
Scheduling	Management commands started by systemd timers	Section 4.1.7
Quality tools	flake8, Locust, OWASP ZAP baseline scan	NFR-MNT-1, NFR-PERF-1, SEC-REQ-6

Data stores

- **PostgreSQL** holds all entities, including those added in Section 4.1.2, and also the audit table, the outbox table and the document-sequence table. Timestamps are stored in UTC and shown in IST (OR-1). Business rules are enforced by the named constraints of Section 4.1.2.
- **File store.** Profile photographs and complaint photos are kept outside the web root under random names and are served only to authorised users (SEC-REQ-9).
- **Log files** are JSON lines kept for 30 days (Section 4.4.3).
- **Backups** are a daily database dump copied to a different machine and kept for 30 days (NFR-SAFE-1).

Deployment view. Figure 3.2 shows how the pieces are placed. The browser reaches Nginx over HTTPS on port 443, and port 80 is redirected. Nginx passes requests to Gunicorn on the same VM, where Django runs the HMS components. PostgreSQL is reached on port 5432 over the private network only (COM-4)

and may be on the same VM or on a separate one (SRS Section 2.4). The only outgoing connections are e-mail to the SMTP relay and the daily backup copy.

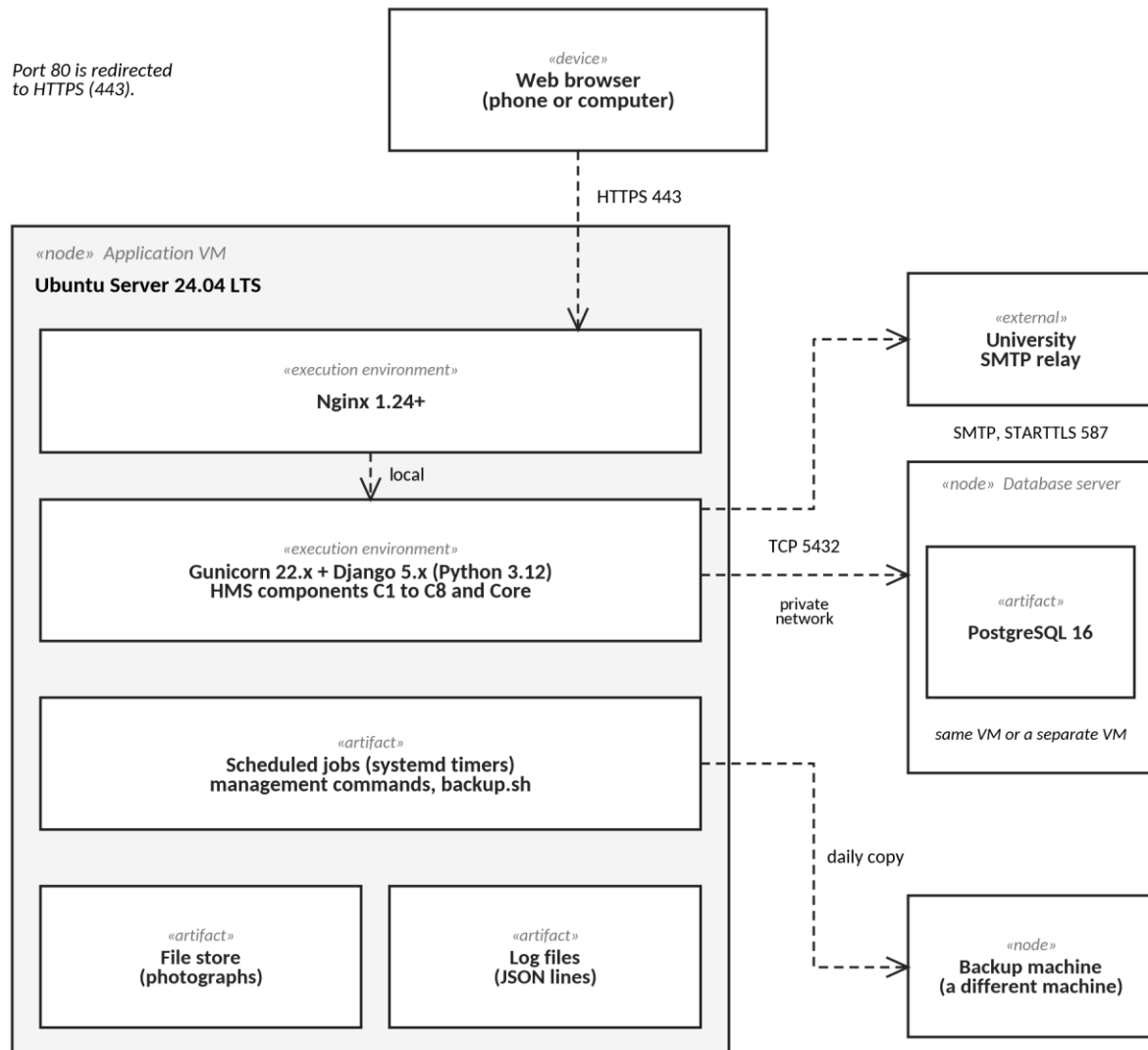


Figure 3.2: Deployment view of HMS

3.7 Risks & Mitigations

#	Risk	Mitigation
1	Two Wardens allocate the same bed at the same time	Room row lock plus partial unique indexes; nightly consistency check (NFR-REL-1)
2	Heavy load on the opening day of the application window	Locust load test before release against NFR-PERF-1; allocation is kept as short transactions
3	Auto-Allocate runs inside a web request and may time out at scale (OI-6)	Measure in the load test; if above 60 s, move it to a management command with no change to the service
4	The SMTP relay is down (A-3)	Outbox retries 3 times at 5-minute intervals, a final failure is logged and alerted; the business action is not blocked
5	Data loss or VM failure	Daily backup to another machine and a monthly restore test (NFR-SAFE-1)

#	Risk	Mitigation
6	One VM is a single point of failure (C-6)	Accepted for release 1.0; backups and the 4-hour recovery target limit the damage
7	Lock-out is used against a victim during the window (OI-7)	Nginx per-address limit; a per-address counter may be added later
8	Personal data leaks (guardian details, addresses)	Scoped access, HTTPS, no sensitive data in logs (Section 3.9)
9	Stale or wrong master list (A-1)	The Administrator imports it before each window; students who are not listed cannot register
10	SRS gaps that are not yet agreed (OI-1 to OI-5, DD-1 to DD-8)	Team review before the SRS and the Test Plan are baselined
11	Look-ups of allocation data by C2, C3, C7 and C8 (arrows 3, 4, 6 and 9) are not yet listed as methods of AllocationService in Section 4.3.3	Keep them read-only and behind C4's service layer; add the read methods to Section 4.3.3 before implementation starts

3.8 Traceability to Requirements

Requirement	Component	Design reference
FR-1.1, FR-1.2	C1 (C2 master list), C8	SD-3 Register
FR-1.3, FR-1.5, FR-1.7	C1	SD-1 Login
FR-1.4, FR-1.8, SEC-REQ-4	C1, Core permissions	Section 4.3.1, 4.3.2
FR-1.6	C1, C8	Section 4.3.2
FR-2.1 to FR-2.4, FR-2.7	C2, Core audit	Section 4.1.2
FR-2.5, FR-2.6	C2	Search API, Section 4.3.5
FR-3.1 to FR-3.7	C3	Section 4.1.2, 4.3.5
FR-4.1 to FR-4.3	C4	SD-2 Part A
FR-4.4 to FR-4.6	C4	SD-2 Part B, Section 4.1.4
FR-4.7	C4	Section 4.1.5
FR-4.8 to FR-4.10	C4, C6, C8	Section 4.3.3
FR-5.1 to FR-5.5	C4	SD-4 Room change
FR-6.1 to FR-6.5	C5	Complaint state model, Section 4.1.3
FR-7.1 to FR-7.6	C6	Payment API, Section 4.3.4
FR-8.1 to FR-8.4	C7	Section 4.3.6
FR-9.1, FR-9.2	C8	Outbox, Section 4.1.7
FR-9.3	C8	Section 4.3.6
NFR-PERF-1 to NFR-PERF-5	All components, deployment view	Section 3.6, 4.4.4
NFR-SAFE-1	Deployment, backup job	Section 4.1.7
NFR-SAFE-2	C4, C3	Single allocate path, Section 4.1.2
NFR-AVL-1, NFR-REL-1	Deployment, scheduled jobs	Section 4.1.7, 4.4.4
SEC-REQ-1 to SEC-REQ-11	Core, C1, Nginx	Section 3.9

3.9 Security Architecture

Trust boundaries. There are three: browser to Nginx, application to PostgreSQL, and application to the SMTP relay. Browser traffic is HTTPS only with TLS 1.2 or higher and HSTS with a max-age of at least 31,536,000 s (SEC-REQ-3, COM-1). The database accepts connections only from the private network (COM-4). Secrets are read from environment variables and are not kept in the repository (SEC-REQ-10).

Main mechanisms

- **Authentication.** Passwords are hashed with PBKDF2-SHA256 (at least 600,000 iterations) or Argon2id and follow the policy in SEC-REQ-2. An account is locked for 15 minutes after 5 failed logins in 10 minutes. The error message is the same for an unknown e-mail and a wrong password. Session cookies are Secure, HttpOnly and SameSite=Lax, the session id is regenerated at login, and a session ends after 30 minutes of inactivity or at logout (SEC-REQ-5).
- **Authorisation.** Every request is checked on the server against BR-1 by Core. An object outside the user's scope returns 404 and is logged, so changing an id in a URL shows nothing (SEC-REQ-4). The last active Administrator cannot be deactivated or demoted (DD-8).
- **Input and output.** A CSRF token is required on every state-changing request, output is HTML-escaped, and all queries go through the ORM (SEC-REQ-6). Uploads are checked by content, limited to 2 MB and to JPEG or PNG, and stored outside the web root (SEC-REQ-9, OI-5).
- **Audit.** The audit table is append-only and is written in the same transaction as the change it records. The runtime database role cannot delete from it, and only the Administrator can view it (SEC-REQ-8, Section 4.4.3).
- **Data protection.** A privacy notice and acknowledgement are shown at registration (OR-3), only the fields of Appendix B are collected, students can download their own data as a PDF (SEC-REQ-11), and records are anonymised 3 years after the last allocation (OR-2). Logs never hold passwords, tokens, session ids or request bodies.

Threat model (STRIDE)

Threat	Examples in HMS	Controls
Spoofing	Password guessing, stolen session	Password policy and hashing, lock-out, e-mail verification, session cookie flags, hashed one-time tokens
Tampering	Forged request, altered allocation or fee	CSRF tokens, server-side authorisation, database constraints, single transaction, HTTPS
Repudiation	A Warden denies an allocation or a payment	Audit log with user, action, old and new values, time and IP address
Information disclosure	Seeing another student's data, leaks through logs	Scope check returning 404, HTTPS and HSTS, output escaping, logs without sensitive data, files outside the web root
Denial of service	Login flooding, reset requests, opening-day overload	Lock-out, 5 reset requests per e-mail per hour, Nginx per-address limit, load test, health probe
Elevation of privilege	A Student calls a Warden URL, a role is changed	Permission matrix on every request, only the Administrator can change roles, one code path for creating allocations

4. Design

4.1 Design Overview

4.1.1 Layering and package structure

Each component is a Django application with four layers of its own. Models hold fields, relationships and database constraints and little else. Services hold business rules, open the transactions, call Core for authorisation and audit, and call other components only through their service interfaces. Forms validate and normalise input. Views and templates only translate between HTTP and the service layer. Three rules follow: no view or template touches the ORM directly except through a service or form; no component writes to another component's tables; and every state change that SRS Appendix C or Section 5.3.2 requires to be audited is audited inside the service, in the same transaction as the change, so a failed audit write rolls back the change.

```
hms/           settings, urls, wsgi
core/          permissions.py, audit.py, middleware.py, exceptions.py, sequences.py, validators.py
accounts/      C1: models, forms, services, views, urls, templates
students/      C2: profile, importers.py (master list), search
rooms/         C3: hostels, rooms, importers.py (rooms CSV)
allocation/    C4: services.py (allocation, auto-allocation, change, check-out), signals.py
complaints/    C5
fees/          C6: services.py, receipts.py
reports/       C7
notifications/ C8: services.py, tasks.py (dispatcher), announcements
templates/     static/ ops/ (backup.sh, systemd units) docs/
```

4.1.2 Data design: additions to SRS Figure 3

SRS Figure 3 shows the main entities but several entities that the functional requirements need are absent, and one relationship conflicts with FR-3.1. The table lists what this design adds or changes. These items should be reflected in the SRS data model when it is next revised (Section 4.6, OI-1).

Entity	Key attributes and constraints	Purpose and SRS reference
MASTER_LIST_ENTRY	srn (unique), name, gender, programme, year, institutional_email (unique), import_batch, imported_at	Source of registration checks and read-only profile fields; A-1, FR-1.1, FR-2.2
HOSTEL_WARDEN	hostel_id, user_id; unique pair	Many-to-many assignment of wardens to hostels; replaces the single warden foreign key of Figure 3; FR-3.1
APPLICATION_WINDOW	academic_year (unique), start_date, end_date, created_by	FR-4.1
STUDENT (added fields)	priority_flag (set by Administrator), privacy_ack_at	BR-4 medical or disability priority; OR-3 acknowledgement
ROOM_APPLICATION (added fields)	application_no (APP-YYYY-NNNNN, unique), roommate_srn (optional)	FR-4.2; Appendix B Table B-3
ALLOCATION (added fields)	academic_year, application_id, end_reason; status in Provisional, Confirmed, Ended, Cancelled	FR-5.5, BR-5, BR-8
ROOM_CHANGE_REQUEST	student_id, from_allocation_id, reason (10 to 300 characters), status (Pending, Approved, Rejected), decision_reason, decided_by, created_at	FR-5.1 to FR-5.3
FEE_RECORD	allocation_id (unique), academic_year, total_fee, status (Unpaid, Partial, Paid); amount paid is the sum of its payments	FR-7.1; no such entity exists in Figure 3

Entity	Key attributes and constraints	Purpose and SRS reference
FEE_PAYMENT (changed)	fee_record_id replaces allocation_id; receipt_no (unique), recorded_by, submission_id (unique, generated when the form is rendered)	FR-7.2, FR-7.4; keyed to the fee record so payments survive a room change (DD-3)
COMPLAINT (added fields)	complaint_no (CMP-YYYY-NNNNN), hostel_id, room_id, photo_path, remark, resolved_at	FR-6.1 to FR-6.4
ANNOUNCEMENT	hostel_id, title (100), body (2,000), expires_on, posted_by	FR-9.3
NOTIFICATION_OUTBOX	kind, recipient_email, payload, entity_id, status (Pending, Sent, Failed), attempts, next_attempt_at, last_error; unique (kind, entity_id, recipient_email)	FR-4.10, FR-9.1, FR-9.2; the unique key makes enqueueing idempotent
AUTH_TOKEN	user_id, purpose (VERIFY or RESET), token_hash (SHA-256, the raw token is never stored), expires_at, used_at	FR-1.2, FR-1.6
AUTH_ATTEMPT	kind (LOGIN_FAILURE or RESET_REQUEST), key (lower-cased e-mail), ip, occurred_at; USER gains locked_until	FR-1.5, SEC-REQ-7
DOCUMENT_SEQUENCE	prefix (APP, CMP, RCP), year, last_no; unique (prefix, year)	Gap-free sequential identifiers for committed records; used instead of PostgreSQL SEQUENCE objects because constraint C-2 forbids hand-written SQL
AUTO_ALLOCATE_RUN	hostel_id, started_by, started_at, finished_at, status (Running, Done, Failed), allocated_count, unallocated_count	FR-4.7 summary; prevents two simultaneous runs for a hostel

The following database constraints implement the rules that the SRS says must be enforced by the database. Each has a fixed name so that the error handler can translate an IntegrityError into the correct user message (Section 4.4.1). Foreign keys use PROTECT so that no record can be removed by cascade (FR-3.5, NFR-SAFE-3). Constraints cannot refer to the current time, so expiry of allocations is handled by the expire_allocations job rather than by a date condition; a failed job therefore errs on the safe side by keeping a bed occupied.

Constraint name	Rule	Enforces
uniq_active_bed	UNIQUE (room_id, bed_no) WHERE status IN (Provisional, Confirmed)	FR-4.6: a bed is never held twice
uniq_active_allocation_per_student	UNIQUE (student_id) WHERE status IN (Provisional, Confirmed)	FR-4.6, BR-3: one active allocation per student
uniq_application_per_year	UNIQUE (student_id, academic_year)	FR-4.3, BR-3
uniq_pending_change_per_student	UNIQUE (student_id) WHERE status = Pending	FR-5.1: one pending request
uniq_running_autoalloc_per_hostel	UNIQUE (hostel_id) WHERE status = Running	FR-4.7: one run at a time
uniq_room_no_per_hostel	UNIQUE (hostel_id, room_no)	FR-3.2
uniq_fee_record_per_allocation, uniq_receipt_no, uniq_payment_submission, uniq_window_per_year	UNIQUE on the named columns	FR-7.1, FR-7.4, FR-7.2 (no duplicate payment), FR-4.1

Constraint name	Rule	Enforces
chk_room_ranges	CHECK floor 0 to 20, capacity 1 to 4, annual_fee 0 to 500000	FR-3.2
chk_bed_no_positive, chk_payment_positive	CHECK bed_no >= 1; CHECK amount > 0	FR-4.6, FR-7.2

Two rules cannot be written as single-table constraints and are therefore enforced in the service layer and verified by tests: that bed_no does not exceed the capacity of the room, and that the hostel's gender designation equals the student's gender (BR-2, NFR-SAFE-2). To keep the second rule reliable there is exactly one code path that creates an Allocation row, the allocate function of AllocationService, and manual allocation, Auto-Allocate and room change all use it.

4.1.3 State models

Entity	States and transitions
Application	Pending to Allocated (manual or automatic allocation); Pending to Rejected (Warden, reason of at least 10 characters); Withdrawn exists in Table B-3 but no function in the SRS produces it (OI-4).
Allocation	Provisional to Confirmed when the fee record becomes Paid (FR-7.3); Provisional to Cancelled by a Warden when 15 days have passed with no payment (BR-5); Provisional or Confirmed to Ended on check-out, room change or after the end date (31 May, BR-8). Provisional and Confirmed are the active states.
Room change request	Pending to Approved or Rejected.
Complaint	Open to In Progress to Resolved; Open or In Progress to Rejected; Resolved back to Open when the student reopens within 7 days (DD-7).
Fee record	Unpaid to Partial to Paid, derived from the balance after each payment.
Room	Available and Full are computed from occupancy and capacity after every allocation change; Under Maintenance is set manually and is never changed by the computation (FR-3.3, FR-3.4).
Outbox message	Pending to Sent; Pending stays Pending with next_attempt_at advanced by 5 minutes after a failed attempt, up to 3 retries after the first attempt; then Failed and logged (FR-9.2).

4.1.4 Allocation and concurrency control

Allocating a bed is the most contended operation. The allocate function runs inside one transaction. It first locks the target room row with select_for_update, which serialises all allocations into that room. It then re-reads the application (which must still be Pending), checks that the student has no active allocation, that the hostel's gender designation matches the student (BR-2), that the room is not Under Maintenance, that the hostel is in the Warden's scope and that the requested bed number is within capacity and not held. It then inserts the Allocation as Provisional, marks the application Allocated, recomputes the room status, raises the AllocationCreated signal that makes C6 create the fee record, enqueues the notification and writes the audit entry. If any step fails, the whole transaction is rolled back.

The lock and the constraints serve different purposes. The lock lets a second Warden who clicks Allocate at the same moment wait a few milliseconds and then receive the clear message Bed no longer available, and it keeps the transaction within the 1-second target of NFR-PERF-2. The partial unique indexes guarantee correctness even if a future code path forgets the lock, as FR-4.6 requires; a violated index is caught as IntegrityError and translated to the same message. When a room change must lock two rooms, they are locked in ascending room identifier order so that two simultaneous changes cannot deadlock.

PostgreSQL's default READ COMMITTED isolation is sufficient because correctness rests on locks and constraints and not on snapshot behaviour.

4.1.5 Auto-Allocate algorithm

Auto-Allocate (FR-4.7) first inserts an AUTO_ALLOCATE_RUN row with status Running for the hostel, which the unique constraint refuses if another run is active. It then loads the pending applications of students whose gender matches the hostel, ordered as in FR-4.4: first-year students, then students with the Administrator-set priority flag, then all others, and within each group by submission time, oldest first. It builds an in-memory map of vacant beds per room type for rooms that are not Under Maintenance, ordered by floor, room number and bed number so that results are deterministic. For each application in order it tries the student's preferences in priority order and picks the first vacant bed of that room type; if a preferred roommate already holds or has just been given a bed in a room of the wanted type with a vacancy, that room is tried first as a best-effort courtesy, not a guarantee. Each assignment calls the same allocate function in its own short transaction, so a conflict affects only that application: on BedUnavailable the map entry is dropped and the next candidate is tried. Applications for which no preference can be satisfied remain Pending and are listed as unallocated. The run row is finally updated and the summary of allocated and unallocated applications is shown. With one transaction per application, 500 applications fit the 60-second target of NFR-PERF-4 if each takes under about 120 ms, comfortably above the 1-second per-allocation bound of NFR-PERF-2.

4.1.6 Design decisions on SRS ambiguities

The following decisions were needed because the SRS is silent, ambiguous or inconsistent. Each is a proposal for the team to confirm; the SRS should be amended where marked.

ID	Question in the SRS	Decision in this design	SRS change?
DD-1	Applications carry room-type preferences but no hostel, yet FR-2.6, FR-4.4 and FR-4.7 speak of a Warden's hostel's applications.	An application is in a Warden's scope when the student's gender matches the gender designation of at least one hostel assigned to that Warden. Auto-Allocate for a hostel considers pending applications of matching gender.	Yes: state the rule in FR-4.4
DD-2	FR-3.1 allows one or more Wardens per hostel; Figure 3 has one warden foreign key.	HOSTEL_WARDEN association table.	Yes: fix Figure 3
DD-3	FR-7.1 creates a fee record per allocation, but a room change creates a new allocation, which would bill the student twice.	The existing fee record is re-pointed to the new allocation, its total set to the new room's annual fee, payments are kept. The new allocation is Confirmed if amount paid is at least the new total, otherwise Provisional. No refund is modelled (A-4).	Yes: add to FR-5.2 and FR-7.1
DD-4	FR-1.1 rejects registration if an account exists for the SRN; an unverified account whose link has expired would block the student for ever.	Only an activated account counts as existing. Registering again for an unverified SRN re-issues the verification link after re-checking the master list.	Yes: clarify FR-1.1
DD-5	The use case Apply for Room includes Check Room Availability, but FR-3.6 defines the grid for staff.	Students see only the number of vacant beds per room type in hostels matching their gender; room numbers are shown to staff only.	Optional
DD-6	BR-8 ends allocations on 31 May; the Allocation entity has start and end dates and a status.	end_date is set to the next 31 May at creation and overwritten with the actual date on check-out or room change; status Ended is set by expire_allocations; Cancelled is used for BR-5 cancellations.	Optional

ID	Question in the SRS	Decision in this design	SRS change?
DD-7	FR-6.4 lets a student reopen a Resolved complaint but not the resulting status.	A reopened complaint returns to Open.	Optional
DD-8	The Administrator may deactivate any account (FR-1.8).	The last active Administrator cannot be deactivated or demoted, so the system cannot lock itself out.	Optional

4.1.7 Scheduled jobs

Time-driven actions are Django management commands started by systemd timers on the application VM, so no message broker is needed. The outbox table is the queue for mail, and the commands below are the only background processing in release 1.0.

Job	Schedule	Purpose	SRS
send_pending_notifications	Every minute	Send outbox e-mails that are due; retry failed ones 3 times at 5-minute intervals; log final failures	FR-4.10, FR-9.1, FR-9.2
expire_allocations	Daily 00:05 IST	Set status Ended on active allocations whose end date has passed (31 May rule)	BR-8
nightly_consistency_check	Daily 01:30 IST	Verify zero beds with more than one active allocation and zero rooms whose stored status differs from computed occupancy; e-mail the Administrator on any finding	NFR-REL-1
backup (ops/backup.sh)	Daily 02:00 IST	Dump the database and copy it to a different machine; keep 30 days	NFR-SAFE-1
restore_test	Monthly	Restore the latest backup into a scratch database and run the consistency check	NFR-SAFE-1
anonymise_alumni	Monthly	Anonymise records of students whose last allocation ended more than 3 years ago	OR-2

4.2 UML Sequence Diagrams

Four sequence diagrams cover the flows to which SRS Appendix C already refers: SD-1 Login, SD-2 Apply and Allocate, SD-3 Register and SD-4 Room change. They were chosen because they exercise the security-critical path (login and lock-out), the most contended write (allocation), the cross-component registration flow, and the atomic multi-entity transaction (room change). In all diagrams a mail is queued in the same transaction as the business change and delivered later by the dispatcher.

4.2.1 SD-1 Login with lock-out

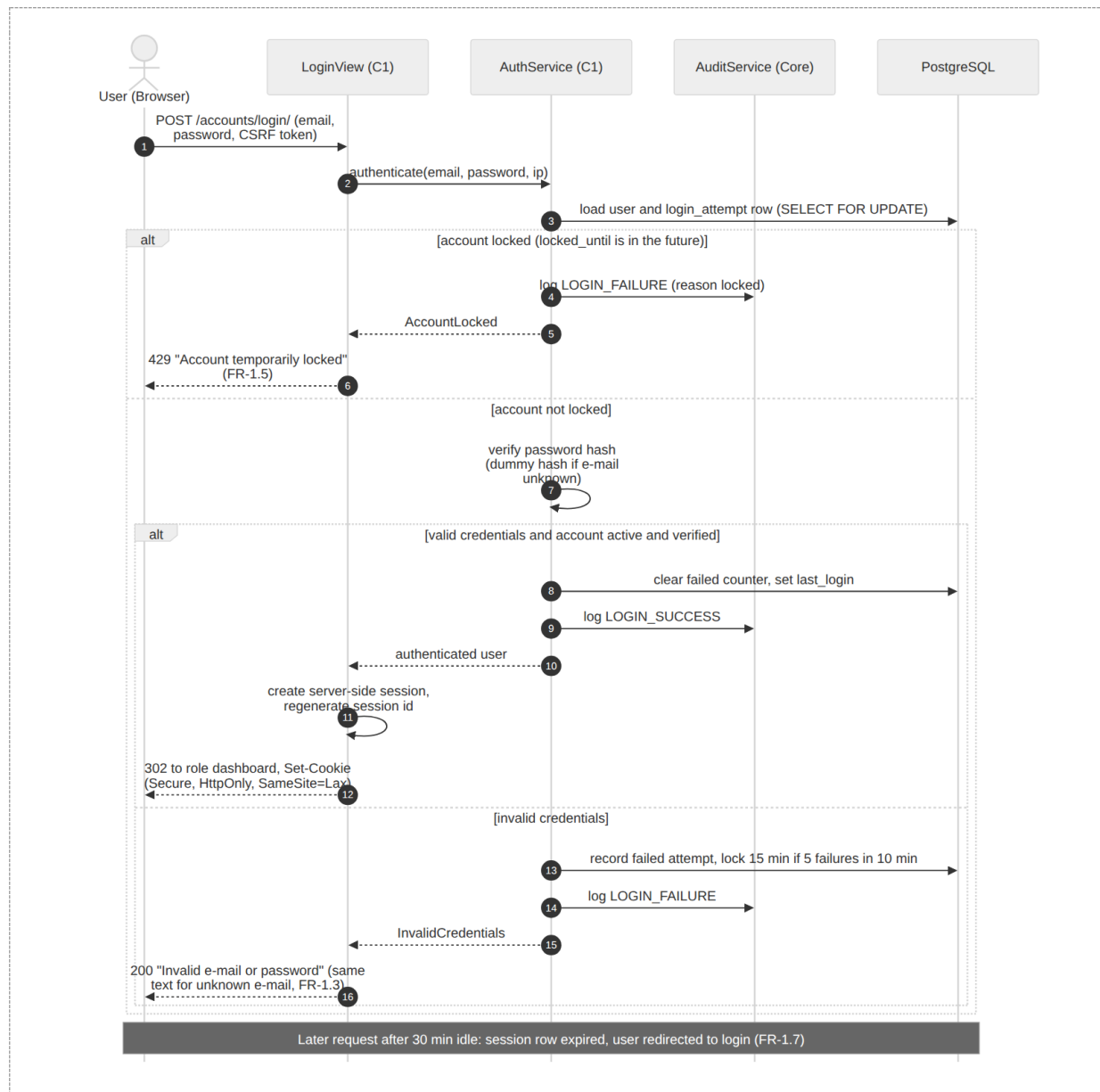


Figure 4.1: SD-1 Login with lock-out and session creation

The authentication service loads the user's attempt state with a row lock so that two parallel failed logins cannot both read a count of 4 and slip past the limit of 5. The same generic message is returned for an unknown e-mail and for a wrong password, and a dummy hash is computed for unknown e-mails so that response time does not distinguish the two cases. Every outcome is audited. This diagram realises FR-1.3, FR-1.5, FR-1.7, SEC-REQ-5, SEC-REQ-7 and SEC-REQ-8.

4.2.2 SD-2 Apply and Allocate

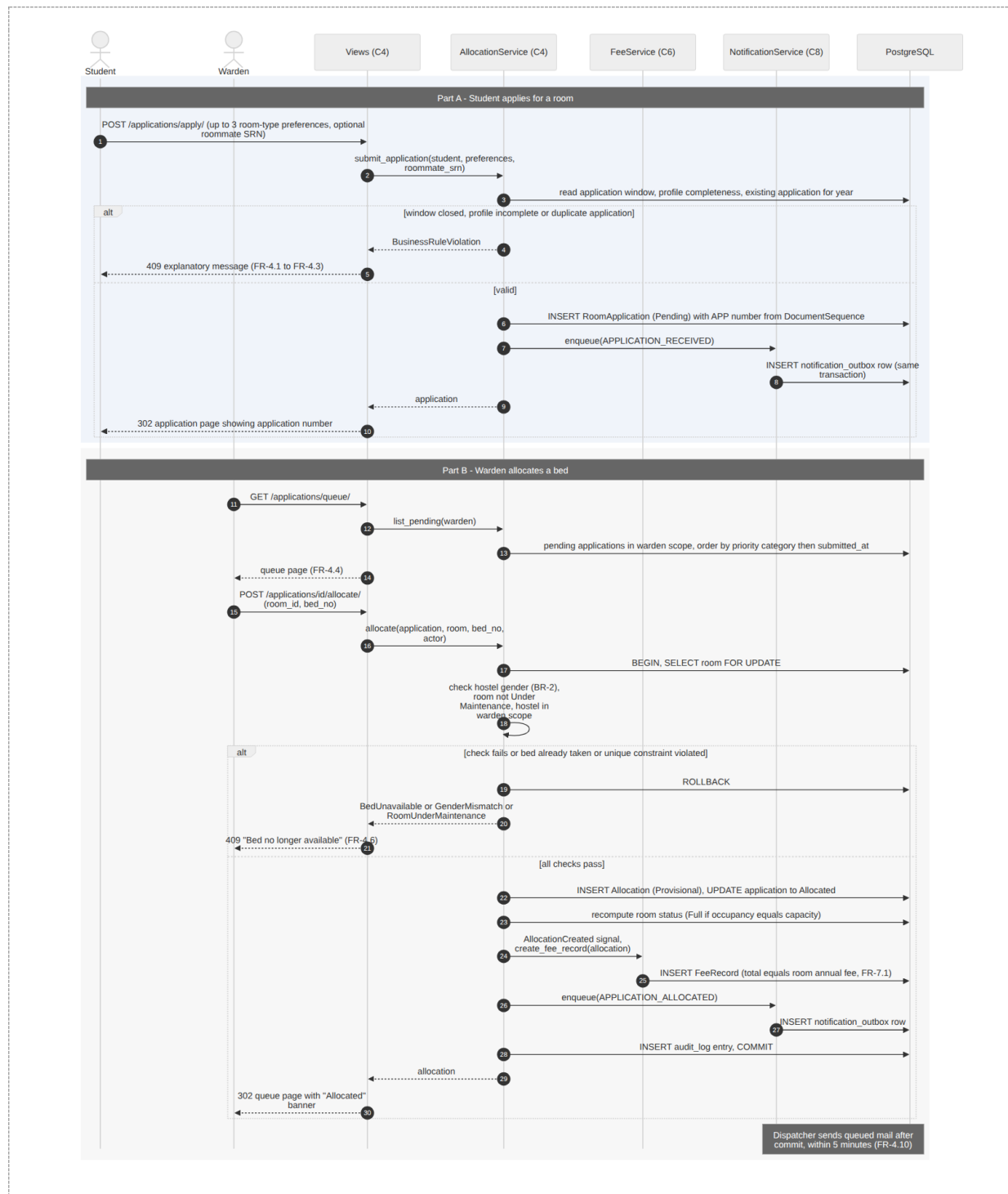


Figure 4.2: SD-2 Student applies, Warden allocates a bed

Part A shows the application checks of FR-4.1 to FR-4.3 and the numbering through DocumentSequence. Part B shows the allocation transaction of Section 4.1.4: the room lock, the checks, the Provisional allocation, the status recomputation, the fee record created by C6 through the AllocationCreated signal, the queued mail and the audit entry, all committed together or not at all. The failure branch is the second Warden of stimulus 5 in SRS Section 4.4.2, who receives Bed no longer available. This diagram realises FR-4.1 to FR-4.6, FR-7.1, FR-4.10 and NFR-SAFE-2.

4.2.3 SD-3 Register

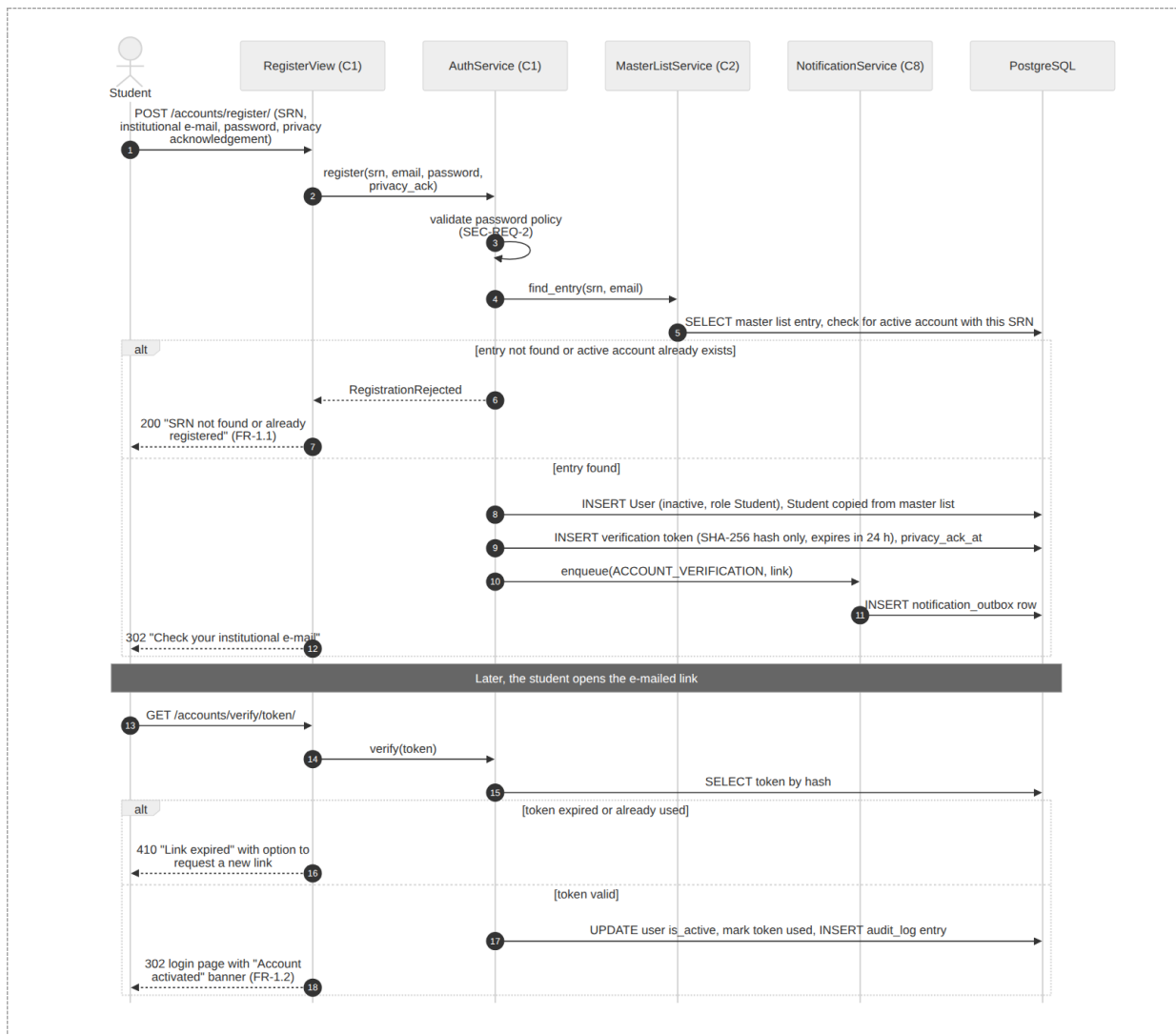


Figure 4.3: SD-3 Student self-registration and e-mail verification

Registration crosses two components: C1 validates the password and creates the account, while C2 owns the master list. The account is created inactive, and only the SHA-256 hash of the verification token is stored, so a database leak does not yield usable links. Activation happens only when the link is opened within 24 hours. This diagram realises FR-1.1, FR-1.2, FR-9.1, SEC-REQ-2 and OR-3, and the re-registration rule of DD-4.

4.2.4 SD-4 Room change



Figure 4.4: SD-4 Room change request and atomic approval

The request step enforces one pending request and the 30-day rule of BR-6. Approval locks both rooms in ascending identifier order, ends the old allocation and creates the new one in the same transaction, moves the fee record (DD-3) and recomputes both room statuses. Because the old allocation is ended before the new one is inserted, the per-student unique index is never violated inside the transaction, and if anything fails the student keeps the old bed. This diagram realises FR-5.1 to FR-5.3, BR-6 and NFR-SAFE-2.

4.3 API Design

4.3.1 Conventions

HMS exposes three kinds of interface. HTML endpoints are server-rendered pages and form submissions; they use form-encoded POST with a CSRF token, answer a successful change with a 302 redirect and a banner message (UI-5), and answer invalid input with a 200 response that re-renders the form with field errors (UI-3). JSON endpoints are read-only GET requests used by the browser for live data and are the only endpoints described by JSON bodies. Service interfaces are the Python methods through which components call one another; they are the contracts that unit tests target. Roles are Student, Warden and Admin. For Warden, every endpoint is further limited to the Warden's hostels (BR-1). Object-level access outside the user's scope returns 404 and not 403, so that the existence of another student's record is not revealed (SEC-REQ-4). Business-rule rejections return 409, rate limiting returns 429 with a

Retry-After header, and an expired or used one-time link returns 410. Section 4.4.1 gives the full mapping.

The JSON error envelope is the same for every JSON endpoint:

```
{ "error": { "code": "VALIDATION_ERROR",
            "message": "Invalid filter value",
            "fields": { "floor": "Must be between 0 and 20" } } }
```

4.3.2 C1 Auth interface

Method and path	Roles	Request fields	Success	Errors
POST /accounts/register/	Public	srn, email, password, password_confirm, privacy_ack (must be true)	302 to /accounts/register/done/	200 field errors; 200 SRN not found or already registered (FR-1.1)
GET /accounts/verify/{token}/	Public	token in path	302 to login with banner	410 link expired or already used
POST /accounts/login/	Public	email, password	302 to role dashboard	200 Invalid e-mail or password; 429 account temporarily locked
POST /accounts/logout/	Any signed-in user	none	302 to login; session deleted on the server	405 for GET, so a link cannot log a user out
POST /accounts/password-reset/	Public	email	200 with the same neutral message whether or not the account exists	429 after 5 requests per e-mail in 1 hour
POST /accounts/password-reset/{token}/	Public	new_password, confirm	302 to login; the user's other sessions are invalidated	410 expired (30 minutes) or used; 200 policy errors
POST /manage/users/	Admin	email, role, hostels (for Warden)	302 to user list; e-mail with a set-password link	409 duplicate e-mail; 403
POST /manage/users/{id}/role/ and /deactivate/	Admin	role, or none	302 to user list	409 last active Administrator; 404

4.3.3 C4 Allocation interface

The HTTP endpoints of C4 are listed first, followed by the service interface that they call and that C6 and C5 also use.

Method and path	Roles	Request fields	Success	Errors
POST /applications/apply/	Student	pref_1 (required), pref_2, pref_3 (distinct room types), roommate_srn (optional)	302 to /applications/{application_no}/	409 window closed, duplicate application, profile incomplete; 200 field errors
GET /applications/queue/	Warden, Admin	query: year, pref, page	200 list ordered by priority then time (FR-4.4)	403
POST /applications/{id}/allocate/	Warden, Admin	room_id, bed_no	302 to queue with Allocated banner	409 Bed no longer available, gender mismatch, room under maintenance, student already allocated; 404 out of scope
POST /applications/{id}/reject/	Warden, Admin	reason (at least 10 characters)	302 to queue	200 reason too short; 404
POST /applications/auto- allocate/	Warden, Admin	hostel_id	200 summary of allocated and unallocated applications	409 a run is already active; 404 hostel not assigned
POST /changes/request/	Student	reason (10 to 300 characters)	302 with banner	409 no Confirmed allocation, request pending, within 30 days
POST /changes/{id}/approve/	Warden, Admin	room_id, bed_no	302 with banner	409 Bed no longer available or gender mismatch; 404
POST /changes/{id}/reject/	Warden, Admin	reason	302 with banner	200 reason missing; 404
POST /allocations/{id}/check- out/	Warden, Admin	date (not after today), override_reason (required if fee balance is not zero)	302 with banner; bed released at once	409 balance outstanding without reason (BR-7); 200 date in the future

AllocationService method	Parameters	Returns	Raises
submit_application	student, preferences (1 to 3 distinct room types), roommate_srn	RoomApplication (Pending)	WindowClosed, ProfileIncomplete, DuplicateApplication

AllocationService method	Parameters	Returns	Raises
list_pending	actor, filters	Queryset in actor's scope, ordered per FR-4.4	AccessDenied
candidate_beds	application, actor	Vacant beds in gender-matching, available rooms	AccessDenied, ObjectNotFound
allocate	application, room, bed_no, actor	Allocation (Provisional)	BedUnavailable, GenderMismatch, RoomUnderMaintenance, AlreadyAllocated, AccessDenied
auto_allocate	hostel, actor	Summary with allocated and unallocated lists	RunInProgress, AccessDenied
reject	application, reason, actor	RoomApplication (Rejected)	ValidationFailure, AccessDenied
request_room_change	student, reason	RoomChangeRequest (Pending)	NoConfirmedAllocation, RequestPending, TooEarly, ValidationFailure
approve_room_change	request, room, bed_no, actor	New Allocation	BedUnavailable, GenderMismatch, RoomUnderMaintenance, AccessDenied
check_out	allocation, date, override_reason, actor	Allocation (Ended)	BalanceOutstanding, ValidationFailure, AccessDenied
confirm	allocation	Allocation (Confirmed); called only by C6	InvalidTransition
cancel_unpaid	allocation, actor	Allocation (Cancelled)	NotCancellable, AccessDenied

4.3.4 C6 Fees interface

Method and path	Roles	Request fields	Success	Errors
POST /fees/allocations/{allocation_id}/payments/	Warden, Admin	amount (greater than 0), mode (Online transfer, Demand draft, Cash), txn_reference (required unless Cash), paid_on (not in the future), submission_id (hidden)	302 to the receipt; balance updated; fee record Paid and allocation Confirmed when balance reaches zero	200 field errors including amount exceeds outstanding balance (FR-7.3); 404 out of scope
GET /fees/payments/{receipt_no}/receipt.pdf	Student (own), Warden, Admin	receipt_no in path	200 application/pdf	404 if not own or not in scope
GET /fees/my/	Student	none	200 total fee, paid, balance and payment history (FR-7.5)	403
GET /fees/provisional-unpaid/	Warden, Admin	none	200 Provisional allocations with no payment after 15 days (FR-7.6)	403

4.3.5 JSON endpoints (C3 and C2)

GET /api/rooms/availability/ serves the room availability grid (FR-3.6) and the availability check of the Apply for Room use case. Query parameters are hostel, floor and room_type, all optional. For staff the response lists rooms in scope; for a student (DD-5) it lists only vacancy counts per room type for gender-matching hostels.

```
GET /api/rooms/availability/?hostel=3&floor=2&room_type=Double      (Warden)
200 { "generated_at": "2026-10-05T10:15:00+05:30",
      "rooms": [ { "room_id": 41, "hostel_id": 3, "room_no": "214", "floor": 2,
                   "room_type": "Double", "capacity": 2, "occupied": 1,
                   "vacant": 1, "status": "Available" } ] }
```

```
GET /api/rooms/availability/                                       (Student)
200 { "generated_at": "...",
      "vacancy_by_type": { "Single": 4, "Double": 17, "Triple": 9 } }
```

Errors: 400 VALIDATION_ERROR (floor outside 0 to 20, unknown room_type),
401 not signed in, 404 hostel outside the caller's scope.

GET /api/students/search/ serves the Student Search screen (FR-2.5). The query q must be at least 2 characters; hostel, room and year are optional filters; results are limited to the Warden's hostels (FR-2.6) and returned 25 per page.

```
GET /api/students/search/?q=pes1ug24&hostel=3&year=2&page=1
200 { "count": 37, "page": 1, "page_size": 25,
      "results": [ { "srn": "PES1UG24CS361", "name": "Raina Kareem",
                    "hostel": "Sarayu", "room": "214",
                    "mobile": "9XXXXXXX", "guardian_mobile": "9XXXXXXX" } ] }
```

Errors: 400 VALIDATION_ERROR (q shorter than 2 characters), 401, 403 (Student role).

The hostel name and the masked numbers above are illustrative values. GET /healthz, restricted by Nginx to the local host and the monitoring probe, returns 200 with {"status": "ok", "db": "ok"} or 503 when the database check fails.

4.3.6 Remaining routes

Component	Routes	Roles	SRS
C2 Profile	GET and POST /profile/; GET /profile/export.pdf; POST /manage/master-list/import/	Student; Admin for import	FR-2.1 to FR-2.4, SEC-REQ-11, A-1
C3 Rooms	GET and POST /manage/hostels/ and /manage/rooms/; POST /manage/rooms/{id}/deactivate/; POST /manage/rooms/import/; GET /rooms/grid/	Admin; Warden for the grid	FR-3.1 to FR-3.7
C4 Allocation	POST /manage/windows/; GET /allocations/history/; POST /allocations/{id}/cancel/	Admin; Warden; Student for own history	FR-4.1, FR-5.5, BR-5
C5 Complaints	GET and POST /complaints/; POST /complaints/{id}/status/; POST /complaints/{id}/reopen/	Student raises and reopens; Warden changes status	FR-6.1 to FR-6.5
C7 Reports	GET /reports/{occupancy, residents, fee-dues, complaints}/?hostel=&from=&to=&format=html, csv or pdf; GET /dashboard/	Warden, Admin; Student dashboard	FR-8.1 to FR-8.4
C8 Notifications	GET /announcements/; POST /announcements/post/	Student reads; Warden posts	FR-9.3
Core	GET /manage/audit-log/	Admin	SEC-REQ-8

4.4 Error Handling, Logging & Monitoring

4.4.1 Error classification and mapping

Services signal failure by raising exceptions from one hierarchy defined in `core/exceptions.py`: `ValidationFailure`, `AccessDenied`, `ObjectNotFound`, `BusinessRuleViolation` (with the specific subclasses named in Section 4.3.3), `RateLimited`, `TokenExpired` and `InvalidUpload`. A central handler in the middleware translates them to responses, so that views contain no error-to-status logic. Constraint violations are translated by constraint name, for example `uniq_active_bed` to `BedUnavailable`. The messages quoted from the SRS are used verbatim.

Condition	Exception	HTTP and presentation	User message	Log level
Field validation fails	<code>ValidationFailure</code>	200 re-render (HTML); 400 (JSON)	Field-level text, for example Enter a valid 10-digit mobile number	None
Not signed in	<code>Authentication required</code>	302 to login (HTML); 401 (JSON)	Please sign in	INFO
Role or hostel not permitted	<code>AccessDenied</code>	403	You do not have permission to do this	WARNING and security event
Object missing or outside scope	<code>ObjectNotFound</code>	404	Not found	WARNING when it is a scope violation
Bed taken or unique index violated	<code>BedUnavailable</code>	409	Bed no longer available	INFO
Gender mismatch	<code>GenderMismatch</code>	409	Student and hostel gender do not match	WARNING (NFR-SAFE-2 attempt)
Other rule violated (window, duplicate, 30-day rule, active allocations)	<code>BusinessRuleViolation</code> subclasses	409	Rule-specific text, for example Room has active allocations	INFO
Payment exceeds balance	<code>ValidationFailure</code>	200 field error	Amount exceeds the outstanding balance of INR x	INFO
Lock-out or request limit	<code>RateLimited</code>	429 with Retry-After	Account temporarily locked / Too many requests	WARNING
Expired or used link	<code>TokenExpired</code>	410	This link has expired	INFO
Bad upload	<code>InvalidUpload</code>	200 field error	Upload a JPEG or PNG of at most 2 MB	WARNING if content does not match type
CSRF check fails	<code>Django CSRF failure</code>	403	Your session expired, reload the page	WARNING
Database unavailable	<code>OperationalError</code>	503	Service temporarily unavailable	CRITICAL
Unexpected error	Any other exception	500 generic page	Something went wrong. Reference: request id	ERROR with stack trace

Condition	Exception	HTTP and presentation	User message	Log level
Mail send fails	SMTP error	No effect on the user's request	None	WARNING per attempt, ERROR after the third retry

4.4.2 Transactions and failure behaviour

Every service method that changes data runs inside `transaction.atomic`. An exception that leaves the method rolls back everything, including the audit entry and the outbox row, so the system never reports an action in its log or mail that did not happen. After a successful POST the view redirects (post-redirect-get), which together with the unique constraints makes an accidental double submit harmless: the second attempt hits `uniq_application_per_year` or `uniq_pending_change_per_student` and is shown as a rule violation instead of creating a duplicate. Payment forms carry a `submission_id` generated when the form is rendered and protected by the unique constraint `uniq_payment_submission`, so a double click cannot record the same payment twice. Outbox enqueueing is idempotent through the unique key on kind, entity and recipient. The dispatcher marks a row Sent only after the relay accepts the message, so a crash can cause at most a duplicate mail and never a lost one.

4.4.3 Logging

Three records are kept. The application log is a rotated file of JSON lines with timestamp in UTC, level, logger, request id (taken from the X-Request-Id header that Nginx sets), user id, path, status and duration, kept for 30 days. The security log records authorisation denials, CSRF failures, lock-outs and rate-limit hits, and is written to the application log with a security marker and also to the audit table where SEC-REQ-8 applies. The audit log is the database table of SEC-REQ-8, kept for at least one year and visible only to the Administrator; entries older than a year may be removed by a separate maintenance role, since the runtime role cannot delete. Logs never contain passwords, tokens, session ids, the Django secret key, guardian or address data, uploaded content or request bodies; users are identified by id and not by e-mail. Final e-mail failures are logged as FR-9.2 requires. Stack traces appear only in the server log and never in a response.

4.4.4 Monitoring

Monitoring uses the same free tools as the rest of the system and no additional infrastructure. Alerts are sent by e-mail to the Administrator through the same SMTP relay; because a failed relay would also stop alerts, the nightly job additionally writes its result to the application log.

Indicator	Source	Alert or target	SRS
Page latency	Nginx access log field <code>request_time</code>	95th percentile above 2 s over 15 minutes, any request above 5 s; Locust before release	NFR-PERF-1
Allocation transaction time	Application log field <code>allocation_ms</code>	95th percentile above 1 s	NFR-PERF-2
Availability	Probe of <code>/healthz</code> every minute	Monthly availability below 99% from 07:00 to 23:00 IST, or below 99.5% in the application window	NFR-AVL-1
Mail backlog	Outbox table	Any Pending message older than 5 minutes; any Failed message	FR-4.10, FR-9.2

Indicator	Source	Alert or target	SRS
Data consistency	nightly_consistency_check exit status	Any bed with more than one active allocation or any room status mismatch	NFR-REL-1
Backups	Backup script status and age of last copy	No successful copy within 26 hours; restore test failure	NFR-SAFE-1
Security events	Security log counts per hour	Review when denials or login failures from one address exceed 50 in an hour	SEC-REQ-4, 7

4.5 UX Design

The interface is a set of server-rendered pages with a common layout: a top navigation bar showing only the menu items allowed for the signed-in role, the user's name and Logout, and a footer with a Help link and the hostel office's contact details (UI-1). After login each role lands on its own dashboard (UI-2): the Student sees application status, allocation, dues, open complaints and announcements; the Warden sees vacancies, pending applications and complaints older than 72 hours; the Administrator sees a system summary. Navigation is flat, no more than two levels deep, so that the Warden's emergency resident list is reachable within 3 clicks from the dashboard (NFR-SAFE-4).

Forms mark mandatory fields with an asterisk, validate on submission, show each error in red directly beneath its field with a summary at the top, and keep valid input already entered (UI-3). The Apply for Room form is deliberately short: three room-type selectors with their vacancy counts and one optional roommate field, so that 80% of first-time users can finish within 5 minutes (NFR-USE-1). Destructive actions (reject, check-out, deactivate) open a confirmation modal (UI-4); every action ends in a dismissible success or failure banner (UI-5). Tables of more than 25 rows paginate and sort (UI-6). Dates are shown as DD-MM-YYYY and amounts as INR with two decimals (UI-8).

Bootstrap 5.3 supplies a responsive grid that works at 360 px without horizontal scrolling (UI-7, NFR-PORT-1). For WCAG 2.1 Level AA (NFR-USE-2), the design uses Bootstrap's default contrast-compliant palette, text alternatives on every image, visible focus indicators, labels bound to every input, and full keyboard operation of modals and menus; automated checks with an accessibility linter are part of the test routine and are complemented by a manual keyboard pass. Strings are marked for translation through Django i18n although release 1.0 is English only (OR-4).

4.6 Open Issues & Next Steps

The issues below were found while designing against the SRS. OI-1 to OI-5 concern the SRS itself and should be settled by the team before the SRS is baselined or the Test Plan is finalised; OI-6 and OI-7 are design risks to be validated early in implementation.

ID	Issue	Impact	Proposed resolution
OI-1	SRS Figure 3 lacks the entities listed in Section 4.1.2 and gives each hostel a single warden.	Test cases and implementation will disagree with the ER diagram.	Issue SRS 1.1 with the corrected ER diagram, or state in the SRS that the SAD data design supersedes Figure 3.
OI-2	DD-1, DD-3 and DD-4 resolve genuine gaps (application scope per Warden, fees on room change, re-registration).	Different readers would implement different behaviour.	Team confirms or changes each decision; amend FR-4.4, FR-5.2, FR-7.1 and FR-1.1.

ID	Issue	Impact	Proposed resolution
OI-3	SRS Appendix C rows 15 and 16 have damaged text (NFR-PERF12 s, NFR-SAFE1Daily).	Traceability entries are unreadable.	Correct to NFR-PERF-1 with 2 s, and NFR-SAFE-1.
OI-4	Table B-3 lists status Withdrawn, but no functional requirement allows a student to withdraw an application.	Dead status value; unclear scope.	Either add a withdraw requirement or remove the status in release 1.0.
OI-5	SEC-REQ-9 allows PDF uploads but no functional requirement accepts a PDF upload; NFR-PERF-1 demands 100% of requests within 5 seconds, which no load test can guarantee.	Unused attack surface; unverifiable requirement.	Limit uploads to JPEG and PNG; restate the 5-second bound as the 99th percentile or as a maximum in the load test.
OI-6	Auto-Allocate runs inside a web request and may approach the Unicorn timeout at the scalability limit of 5,000 students.	Possible time-outs for large hostels.	Measure in the Locust run; if above 60 s, move the call to a management command or outbox-style job, with no change to the service.
OI-7	Lock-out is per account and can be used against a victim.	Targeted denial of access during the application window.	Keep the Nginx per-address limit; consider an additional per-address counter in a later release.

Next steps, in order: the team reviews Section 4.1.6 and Section 4.6 and agrees the SRS changes; the Test Plan is updated so that its test cases reference C1 to C8, SD-1 to SD-4 and the constraint names of Section 4.1.2, with the concurrency case (two threads allocating the same bed) and the room-change atomicity case as explicit cases; a walking skeleton of login, one allocation and one queued mail is implemented first to prove the layering, transactions and outbox; Locust, flake8, coverage and the OWASP ZAP baseline scan are then added to the repository as release gates. Possible later enhancements, none of which are required in release 1.0, are a live link to the university records system instead of CSV import, a payment gateway, a mobile application and two-factor authentication for staff.